

INSPECTION-RECONCILE

Administrator Guide

Inspection documentation readiness, with a read-only Quickbase adapter

Version	0.1.0
Date	9 October 2026
Audience	Administrators and data integration developers who install, configure and integrate the tool
Repository	github.com/bochen2029-pixel/inspection-reconcile
Live demo	bochen2029-pixel.github.io/inspection-reconcile
Project page	opnaorta.ai/inspection-reconcile

Synthetic demonstration data. Every example in this guide uses the fictional North Creek project (NC-001). No real organization's data appears anywhere in this guide or in the repository. Quickbase is a trademark of Quickbase, Inc.; this project is not affiliated with or endorsed by Quickbase.

Contents

1	About this guide	4
1.1	Audience and scope	4
1.2	Where the details live	4
2	Architecture	5
2.1	The pipeline	5
2.2	Repository layout	6
2.3	Design principles	6
3	Installation	7
3.1	Requirements	7
3.2	Windows	7
3.3	macOS and Linux	7
3.4	Optional components	8
3.5	Verify the installation	8
4	The requirement pack	9
4.1	Format	9
4.2	Field by field	9
4.3	Versioning and effective_from	10
4.4	Accepted coverage bases	10
4.5	Approval binding: digest or revisions	11
4.6	Safe YAML	11
4.7	Writing a pack for a real project	11
5	The canonical snapshot	12
5.1	Directory layout	12
5.2	manifest.json	13
5.3	project.json	14
5.4	CSV rules	14
5.5	Columns	14
5.6	Value grammars	16
5.7	Coverage claims and basis tokens	16
6	Quickbase	17
6.1	What the adapter does	17
6.2	The field mapping	17
6.3	Normalizing an export	18
6.4	The capture configuration	19
6.5	Tokens and least privilege	20
6.6	How a capture reads	20
6.7	Rate limits and retries	21
6.8	Live verification	21
7	Security model	23

7.1	Read-only by construction	23
7.2	Tokens	23
7.3	Input parsing	23
7.4	Files and paths	23
7.5	The HTML report	23
7.6	Outputs	24
7.7	Data handling	24
8	Outputs and provenance	25
8.1	assessment.json	25
8.2	run-manifest.json	25
8.3	Identities and determinism	25
8.4	comparison.json	25
8.5	The SQL export and cross-check	26
9	Operations	27
9.1	Command reference	27
9.2	Exit codes	28
9.3	Run-error codes	28
9.4	Diagnostics and --log-json	30
9.5	Automation	30
9.6	Continuous integration and the demonstration site	31
9.7	Rebuilding the screenshots and these guides	31
10	Limitations and verification status	32
10.1	What is verified	32
10.2	Known limitations	32
10.3	What is never claimed	32

1 About this guide

1.1 Audience and scope

This guide is for the person who installs `inspection-reconcile`, prepares its inputs and connects it to a source system: a Quickbase administrator, a data integration developer, or whoever runs it on a schedule. It covers the architecture, installation, the requirement pack, the snapshot format, the Quickbase adapter, the security model, the outputs and day-to-day operation.

Reading reports and acting on findings is covered by the [User Guide](#).

1.2 Where the details live

document	role
docs/SPEC.md	the normative specification. Where this guide and the specification differ, the specification wins
docs/decisions.md	every design decision and amendment, appended and never rewritten
docs/limitations.md	what is verified, what is synthetic and what is unsupported
docs/c3-runbook.md	the step-by-step live verification against a Quickbase test app
docs/status.md	the build steps, and which test proves each invariant and acceptance criterion
SECURITY.md	how to report a vulnerability privately
CHANGELOG.md	what changed in each release

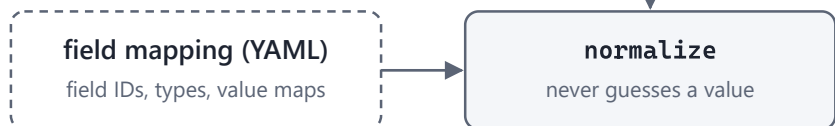
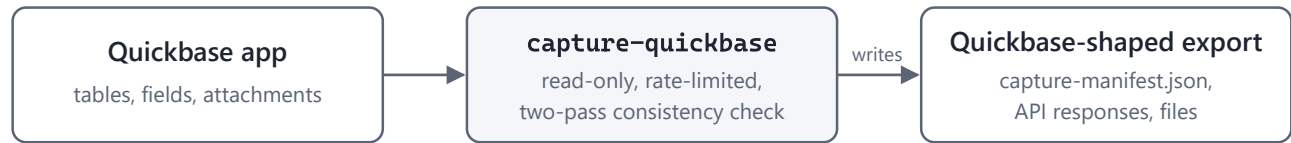
Note

All data in the repository is synthetic: the fictional North Creek project (NC-001). Quickbase is a trademark of Quickbase, Inc.; this project is not affiliated with or endorsed by Quickbase.

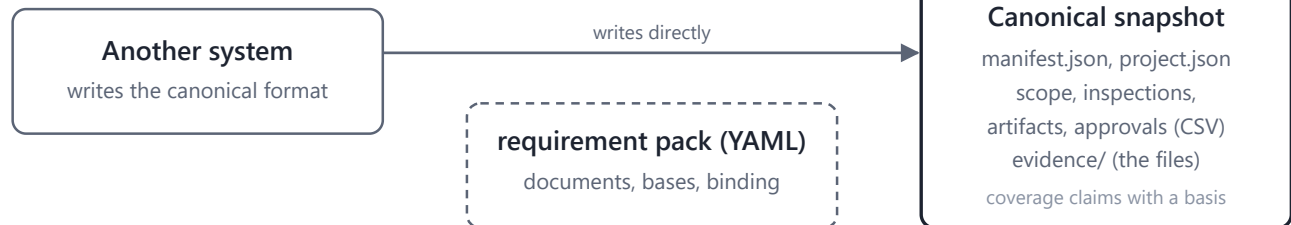
2 Architecture

2.1 The pipeline

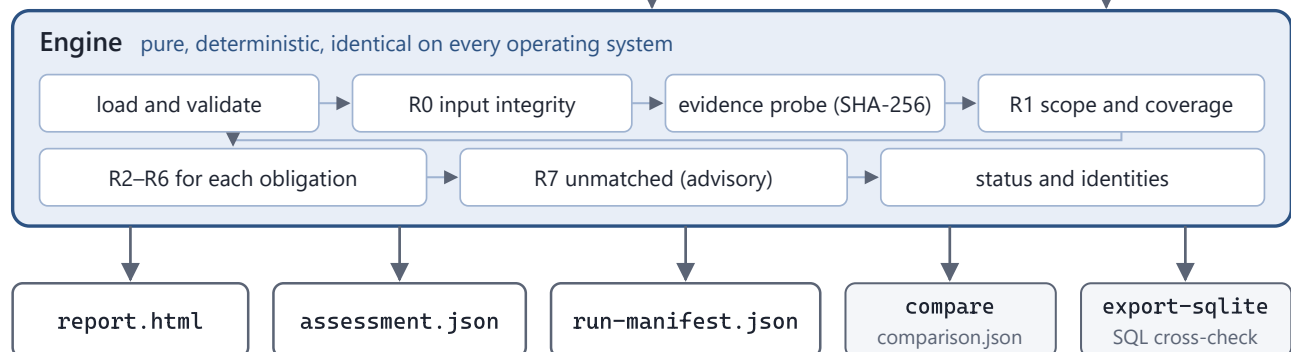
QUICKBASE PATH



ANY OTHER SOURCE



ASSESSMENT



exit code 0 ready for review · 10 blocked · 11 unknown · 2 run error

Figure 1. How data flows: a Quickbase app or any other system produces a canonical snapshot; the engine assesses it against a requirement pack

Two paths lead to the same place, a **canonical snapshot**:

- **From Quickbase**, `capture-quickbase` reads an app through a read-only client and writes a *Quickbase-shaped export*: the raw API responses, the attachments, and a capture manifest. `normalize` then turns the export into a canonical snapshot, using a **field mapping** that says which Quickbase field holds which value.
- **From any other system**, an integration writes the canonical snapshot directly: a manifest, a project file, four CSV files and an evidence folder.

The **engine** reads a snapshot and a **requirement pack** and produces the assessment. It is pure: no clock, no network and no randomness. Evidence files are read only through a probe that hashes them. The same inputs therefore give byte-identical `assessment.json` and `report.html` on every operating system.

2.2 Repository layout

path	contents
<code>src/inspection_reconcile/engine/</code>	the checks R0 to R7, coverage, status and identities
<code>src/inspection_reconcile/io/</code>	snapshot loading, the evidence probe, atomic output writes
<code>src/inspection_reconcile/report/</code>	<code>assessment.json</code> , <code>run-manifest.json</code> , <code>report.html</code> , <code>compare</code> , the reason-code templates
<code>src/inspection_reconcile/adapters/</code>	the Quickbase field mapping, <code>normalize</code> , the read-only HTTP client, the capture
<code>src/inspection_reconcile/cli.py</code>	the command line: arguments, exit codes, the top-level guard
<code>policies/</code> , <code>mappings/</code>	the demonstration requirement packs and the Quickbase field mapping
<code>fixtures/</code>	the oracle (<code>oracle.yaml</code>) and the 24 scenario snapshots and exports
<code>sql/</code>	the SQL cross-check queries
<code>tools/</code>	the fixture generator, the screenshot tool and the guide builder
<code>tests/</code>	unit, scenario, property, fault-injection, CLI, HTML, SQL, adapter and connector tests

2.3 Design principles

- **Never guess.** A defective record is quarantined and reported; an unmapped value is listed, never interpreted.
- **Closed-world inputs.** Every configuration file is validated member by member. An unknown member, a wrong type or an invalid value stops the run with exit code 2 and a message that names the file and the member.
- **Absence needs completeness.** A missing record is a failure only in a dataset whose capture is known to be complete, on a basis the requirement pack accepts.
- **Read-only.** Nothing in the package can write to a source system.
- **Synthetic public data.** Real captures belong in the gitignored `local/` folder.

3 Installation

3.1 Requirements

component	version	needed for
Python	3.12 or 3.13	everything; CI tests both
uv	current	creating the environment and running commands
Git	any	getting the repository
Chrome, Chromium or Edge	current	only to rebuild the screenshots and these guides

Windows, macOS and Linux are all supported. CI runs the full suite on all three, with both Python versions, and checks that every platform produces identical identities and report digests.

3.2 Windows

In PowerShell:

```
winget install --id=astral-sh.uv -e
git clone https://github.com/bochen2029-pixel/inspection-reconcile.git
cd inspection-reconcile
uv sync
```

3.3 macOS and Linux

```
curl -LsSf https://astral.sh/uv/install.sh | sh          # or: brew install uv
git clone https://github.com/bochen2029-pixel/inspection-reconcile.git
cd inspection-reconcile
uv sync
```

uv's own installation guide lists other options, for example `pipx install uv`.

3.4 Optional components

command	installs	needed for
<code>uv sync</code>	PyYAML and Jinja2	every command except <code>capture-quickbase</code>
<code>uv sync --extra quickbase</code>	the above, plus httpx	<code>capture-quickbase</code>
<code>uv sync --all-extras</code>	the above, plus pytest, hypothesis, ruff and mypy	development and the test suite

3.5 Verify the installation

```
uv run inspection-reconcile --version           # inspection-reconcile 0.1.0
uv run inspection-reconcile demo --all --out out/demo  # exit 0: all 24 scenarios match
their oracle
```

On the reference workstation, `demo --all` finishes in about 5 seconds, and a single assessment such as S02, with its 207 findings, in under half a second. The same reports are published at bochen2029-pixel.github.io/inspection-reconcile, and the project page is opnaorta.ai/inspection-reconcile.

With the development extras installed, the full gate is the same one CI runs:

```
uv run pytest -q
uv run ruff check . && uv run ruff format --check .
uv run mypy
```

4 The requirement pack

4.1 Format

A requirement pack is a YAML file that states what a project's documentation must contain. The demonstration pack, `policies/north-creek-demo.yml`:

```
schema: inspection-reconcile/policy/v1
pack_id: north-creek-demo
version: "3.0.0"
effective_from: "2026-09-01"
synthetic: true
project_id: NC-001
scope_revision: S1
coverage:
  accepted_bases:
    - [synthetic_universe]
    - [query_total_matched, two_pass_stable, operator_attestation]
requirements:
  visual_inspection:
    document_kinds: [inspection_report, photo]
approval:
  required_binding: digest
limits:
  max_file_bytes: 104857600
checks:
  - {id: R0, type: input_integrity,      required: true}
  - {id: R1, type: scope_and_coverage,   required: true}
  - {id: R2, type: inspection_cardinality, required: true}
  - {id: R3, type: identity_consistency, required: true}
  - {id: R4, type: required_artifacts,   required: true}
  - {id: R5, type: artifact_availability, required: true}
  - {id: R6, type: approval_binding,     required: true}
  - {id: R7, type: unmatched_records,    required: false}
```

4.2 Field by field

member	rule	meaning
<code>schema</code>	exactly <code>inspection-reconcile/policy/v1</code>	the format version
<code>pack_id</code>	an ID	the pack's name, shown in every report

member	rule	meaning
<code>version</code>	a quoted string	the pack's version; change it whenever a requirement changes
<code>effective_from</code>	a quoted date, <code>YYYY-MM-DD</code>	midnight UTC of that date. An evaluation time before it is a run error (<code>POLICY_NOT_EFFECTIVE</code>)
<code>synthetic</code>	<code>true</code> or <code>false</code>	marks demonstration packs; any synthetic input shows the report's banner
<code>project_id</code>	an ID	must equal the snapshot's <code>project.json</code> (<code>PROJECT_MISMATCH</code> otherwise)
<code>scope_revision</code>	a revision	the scope revision this pack expects; a mismatch is the finding <code>SCOPE_REVISION_MISMATCH</code>
<code>coverage.accepted_bases</code>	a list of lists of basis tokens	each inner list is one acceptable basis set (see below)
<code>requirements</code>	activity → <code>document_kinds</code>	the document kinds each activity needs; a missing activity gives <code>REQUIREMENT_UNDEFINED</code>
<code>approval.required_binding</code>	<code>digest</code> or <code>revisions</code>	how approvals must bind to evidence
<code>limits.max_file_bytes</code>	1 to $2^{31} - 1$	evidence files above this size are not hashed (<code>FILE_TOO_LARGE</code>)
<code>checks</code>	exactly the eight entries shown	fixed by the specification and listed for readability; they cannot be changed

Every member is required, and an unknown member at any level is a run error.

4.3 Versioning and `effective_from`

Treat a pack like a contract. When a requirement changes, give the pack a new `version`, and an `effective_from` date if the change applies from a date. The pack's digest appears in every assessment, and `assessment.json` records the `pack_id` and `version`, so any result can be traced to the exact requirements it was checked against. The digest is computed over the parsed content, so comments and formatting do not change it.

4.4 Accepted coverage bases

A dataset declared `complete_for_declared_scope` counts as complete only when **one** of the pack's accepted basis sets is a **subset** of the basis the dataset declares. With the demonstration pack, a fixture's `[synthetic_universe]` qualifies, and so does a live capture that earned all three of `query_total_matched`, `two_pass_stable` and `operator_attestation`. A manual `[ui_export]` does not, so it gives `COVERAGE_BASIS_NOT_ACCEPTED`.

Warning

Adding a weak basis to `accepted_bases`, such as `ui_export`, turns absences in such captures into established failures. Do it only deliberately, and record why.

4.5 Approval binding: digest or revisions

- **digest** (recommended): an approval must store an *evidence digest*, a SHA-256 over the content of every required file of the inspection. Any later byte change is detected. Approvals that only list revisions are reported as `BINDING_STRENGTH_INSUFFICIENT`.
- **revisions**: an approval may instead list the artifact revisions it covers (`approval_items.csv`). A file replaced under an unchanged revision label goes unnoticed; scenario S18 demonstrates this.

The `evidence-digest` command prints the digest a review process should store with its decision.

4.6 Safe YAML

Every YAML file (packs, mappings, capture configurations) is loaded by a restricted loader:

- duplicate keys, aliases, anchors and explicit tags are rejected;
- a non-string key where a string is expected is rejected. For example, an unquoted `Yes:` parses as a boolean, and the message says to quote it;
- integers are plain decimals only. YAML 1.1 forms such as `0100`, `1:30` or `0x64` load as strings, so a field expecting an integer rejects them instead of silently reading another number.

4.7 Writing a pack for a real project

The demonstration packs are examples. A real pack must state the client's actual requirements: which activities exist, which document kinds each one needs, and what binding approvals use. Agree it with the people who own those requirements. Application configuration cannot establish business intent.

5 The canonical snapshot

Any system can feed the tool by writing a canonical snapshot. Quickbase users normally let `normalize` write it.

5.1 Directory layout

```
<snapshot>/
  manifest.json          required
  project.json           required
  scope.csv              the accepted scope (absent: the finding SCOPE_MISSING, not a run error)
  inspections.csv        as declared in the manifest
  artifacts.csv          as declared in the manifest
  approvals.csv          as declared in the manifest
  approval_items.csv     optional: revision-mode approval bindings
  normalization.json     optional: written by normalize
  evidence/              the evidence root; artifacts' relative_path is relative to it
```

A dataset the manifest declares must exist on disk. A dataset the manifest omits is treated as empty and not complete (`DATASET_MISSING`). The evidence root must be a real folder, not a link or junction.

5.2 manifest.json

```
{
  "format": "inspection-reconcile/snapshot/v1",
  "snapshot_id": "NC-001-S01-clean",
  "synthetic": true,
  "source": {"system": "fixture", "description": "North Creek synthetic baseline"},
  "capture": {"started_at": "2026-10-01T17:55:00Z", "ended_at": "2026-10-01T18:00:00Z"},
  "scope": {
    "file": "scope.csv",
    "scope_revision": "S1",
    "accepted": {"by": "Synthetic Client Records Dept", "at": "2026-08-15T14:00:00Z"},
  },
  "reference": "SYN-SCOPE-S1"}
},
"datasets": {
  "inspections": {
    "file": "inspections.csv", "coverage": "complete_for_declared_scope",
    "basis": ["synthetic_universe"], "consistency": "not_applicable"},
  "artifacts": {
    "file": "artifacts.csv", "coverage": "...", "basis": ["..."],
  },
  "consistency": "...",
  "approvals": {
    "file": "approvals.csv", "coverage": "...", "basis": ["..."],
  },
  "consistency": "...",
  "evidence_files": {"dir": "evidence", "coverage": "...", "basis": ["..."], "consistency":
  "..."}
},
"optional_datasets": {"approval_items": null},
"normalization": null
}
```

member	rule
<code>format</code>	exactly <code>inspection-reconcile/snapshot/v1</code>
<code>snapshot_id</code>	an ID; provenance only
<code>synthetic</code>	boolean
<code>source</code>	<code>{system, description}</code> ; provenance only
<code>capture</code>	<code>started_at</code> ≤ <code>ended_at</code> , both timestamps; <code>ended_at</code> is the default evaluation time
<code>scope</code>	optional: <code>{file, scope_revision, accepted}</code> . Absent gives <code>SCOPE_MISSING</code> ; <code>accepted: null</code> gives <code>SCOPE_NOT_ACCEPTED</code>
<code>datasets.<name>.coverage</code>	<code>complete_for_declared_scope</code> , <code>partial</code> or <code>unverified</code>
<code>datasets.<name>.basis</code>	a non-empty list of distinct basis tokens
<code>datasets.<name>.consistency</code>	<code>not_applicable</code> , <code>stable_verified</code> or <code>changed_during_capture</code>

member	rule
<code>optional_datasets.approval_items</code>	<code>null</code> or <code>{file}</code> ; it shares the approvals coverage
<code>normalization</code>	<code>null</code> , or the name of the normalization file

5.3 project.json

```
{"project_id": "NC-001", "client_id": "CL-SYN-01", "name": "North Creek (synthetic)",
"synthetic": true}
```

All four members are required. `project_id` must equal the pack's.

5.4 CSV rules

- **Encoding.** Strict UTF-8; a leading byte-order mark is removed. Invalid UTF-8 or a NUL byte stops the run.
- **Format.** RFC 4180 quoting, comma-separated, exactly one header row.
- **Header.** Exactly the columns listed below, in any order, each once. A column whose name begins with `x_` is allowed, ignored by the checks and kept in provenance. Any other column stops the run (`CSV_HEADER_INVALID`).
- **Rows.** A row with the wrong number of fields is not a run error: it becomes the finding `MALFORMED_ROW`. An empty cell is `null`. Values are never trimmed. Booleans are exactly `true` or `false`.

5.5 Columns

`scope.csv`, one row per obligation, key `obligation_id`:

column	type	required
<code>obligation_id</code>	ID	yes
<code>project_id</code>	ID	yes
<code>scope_revision</code>	REV	yes
<code>asset_id</code>	ID	yes
<code>activity_kind</code>	KIND	yes

`inspections.csv`, key (`inspection_id`, `revision`):

column	type	required	notes
<code>inspection_id</code>	ID	yes	shared by every revision of one inspection
<code>revision</code>	REV	yes	

column	type	required	notes
<code>is_current</code>	BOOL	yes	at most one current revision per inspection
<code>obligation_id</code>	ID	no	the link to the scope; empty means unlinked
<code>project_id</code> , <code>asset_id</code>	ID	yes	compared with the obligation (R3)
<code>activity_kind</code>	KIND	yes	compared with the obligation (R3)
<code>completion_status</code>	enum	yes	<code>completed</code> , <code>in_progress</code> , <code>not_started</code> or <code>cancelled</code>
<code>completed_at</code>	TS	no	required when the row is current and completed

artifacts.csv, key (`artifact_id`, `revision`):

column	type	required	notes
<code>artifact_id</code>	ID	yes	
<code>revision</code>	REV	yes	
<code>is_current</code>	BOOL	yes	at most one current revision per artifact
<code>inspection_id</code>	ID	yes	links to the inspection, at any revision
<code>project_id</code> , <code>asset_id</code>	ID	yes	compared with the obligation (R3)
<code>document_kind</code>	KIND	yes	matched against the pack's <code>document_kinds</code>
<code>relative_path</code>	PATH	no	the file under the evidence root; empty means no file

approvals.csv, key `approval_id`:

column	type	required	notes
<code>approval_id</code>	ID	yes	
<code>inspection_id</code>	ID	yes	
<code>inspection_revision</code>	REV	yes	the revision the decision covers
<code>evidence_digest</code>	DIGEST	no	the content-digest binding
<code>decision</code>	enum	yes	<code>approved</code> , <code>rejected</code> or <code>revoked</code>
<code>decided_at</code>	TS	yes	orders the decisions
<code>decided_by</code>	TEXT	yes	

approval_items.csv (optional): `approval_id`, `artifact_id` and `artifact_revision`, all required. Each row says that the approval covers one artifact revision.

5.6 Value grammars

type	rule
ID	1 to 128 characters: letters, digits and <code>. _ : -</code> , starting with a letter or digit; compared exactly
REV	1 to 32 characters: letters, digits and <code>. _ -</code> , starting with a letter or digit
KIND	lower case: a letter, then letters, digits or <code>_</code> , up to 64 characters
BOOL	<code>true</code> or <code>false</code>
TS	RFC 3339 with an offset, such as <code>2026-09-03T17:00:00Z</code> or <code>2026-09-03T12:00:00-05:00</code>
DIGEST	<code>sha256:</code> followed by 64 lower-case hex digits
TEXT	1 to 200 characters, with no control characters
PATH	relative, <code>/</code> -separated; no <code>.</code> or <code>..</code> segment; none of <code>\ < > : " ? *</code> ; no Windows device names such as <code>CON</code> or <code>NUL</code> ; at most 240 characters

A value that breaks its grammar is the finding `INVALID_VALUE` (or `INVALID_PATH`), never a run error. The row is quarantined. A path that breaks the PATH rules is never opened.

5.7 Coverage claims and basis tokens

Declare coverage honestly: it decides whether an absence is a failure or an unknown.

basis token	meaning
<code>synthetic_universe</code>	a fixture whose universe is known by construction
<code>query_total_matched</code>	every page was read, and the record count equals the source's reported total
<code>two_pass_stable</code>	a second pass found the same record IDs and modification times
<code>operator_attestation</code>	a named person attests that the reading account can see every record in scope
<code>ui_export</code>	a manual export from an application's user interface
<code>extraction_interrupted</code>	the capture did not finish
<code>attachment_capture_skipped</code>	file contents were deliberately not captured
<code>pagination_incomplete</code>	the connector could not account for every page

The tool computes an *effective* coverage per dataset. It is weaker than the declared one when the basis is not accepted, the source changed during the capture, or the data contradicts the claim, for example a record referring to one the dataset does not contain.

6 Quickbase

6.1 What the adapter does

The Quickbase adapter has three parts:

1. a **field mapping**, verified against the app's live field list by field ID, `fieldType` and mode, never by display label;
2. **normalize**, which turns a Quickbase-shaped export into a canonical snapshot;
3. a **read-only capture client**, which reads an app into such an export.

Important

The adapter was built from Quickbase's published RESTful JSON API contract, checked on 2026-10-09, and is tested against a mock app that follows it. A capture of that mock app gives the same finding for every requirement as the canonical scenario. It has **not yet run against a live Quickbase app**. That step, C3, needs an authorized test app and a user token; see [Live verification](#).

6.2 The field mapping

The mapping says which field of which table holds each canonical column. An excerpt of

`mappings/quickbase-demo.yml`:

```
schema: inspection-reconcile/mapping/v1
mapping_id: quickbase-demo
version: "3.0.0"
source: quickbase
synthetic: true
project: {project_id: NC-001, client_id: CL-SYN-01, name: "North Creek (synthetic)", synthetic: true}
scope_filter: {fid: 7, value: NC-001}          # obligations table only: {7.EX.'NC-001'}
tables:
  inspections:
    table_id: bsyn00002
    fields:
      inspection_id: {fid: 6, type: text}
      revision:      {fid: 7, type: numeric, as: integer_string}
      is_current:    {fid: 8, type: checkbox}
      obligation_id: {fid: 9, type: reference, target: obligations}
      completion_status: {fid: 13, type: text-multiple-choice,
        values: {"Complete": completed, "In Progress": in_progress,
          "Not Started": not_started, "Cancelled": cancelled}}
      completed_at:  {fid: 14, type: timestamp}
```

The mapping types, and the `getFields` field type each one expects:

mapping type	expected fieldType	conversion
<code>text</code>	<code>text</code>	the string; an empty string becomes null
<code>text-multiple-choice</code>	<code>text-multiple-choice</code>	the label, through the <code>values</code> map
<code>numeric</code> with <code>as: integer_string</code>	<code>numeric</code>	an integral number as a decimal string
<code>checkbox</code>	<code>checkbox</code>	<code>true</code> or <code>false</code>
<code>timestamp</code>	<code>timestamp</code>	the string, validated by R0
<code>user</code> with <code>as: email</code>	<code>user</code>	the user's email
<code>file</code>	<code>file</code>	the highest file version, copied into the evidence folder
<code>reference</code>	<code>numeric</code>	the Record ID of a parent record, resolved to the parent's canonical ID
<code>recordid</code>	<code>recordid</code>	the Record ID as a decimal string

Rules worth knowing:

- **Every mapped field must be a data-entry field.** A lookup, summary or formula field is refused, unless the table declares `allow_derived: true`.
- **Value maps are exact and case-sensitive.** A label missing from the map is never guessed. The cell keeps the raw label, `normalization.json` lists it, and R0 reports `UNMAPPED_VALUE`. Scenario S17 shows this.
- **Unexpected JSON types are never coerced.** For example, a number where text is expected is listed as unmapped too.
- **The scope filter applies to the obligations table only.** Child tables are read in full, so a record linked to the wrong project is still seen, and R3 can flag it.

6.3 Normalizing an export

```
uv run inspection-reconcile normalize \
  --export fixtures/scenarios/S16-quickbase-clean/export \
  --mapping mappings/quickbase-demo.yml --out out/s16-snapshot
```

`normalize` checks the export against its own manifest: page files, record counts, file sizes and digests. It refuses an inconsistent export (`EXPORT_INVALID`). It writes the canonical CSV files, `project.json` from the mapping, the coverage declarations from the capture manifest, the evidence files under `evidence/files/<table>/<rid>/<fid>/v<n>/`, and `normalization.json`. That file lists the unmapped values, the unresolved references and the original file names. Every row carries `x_source = "table=<table_id>;rid=<rid>"`, so a finding's evidence points at the exact Quickbase record.

`assess --export` runs the same normalization into a temporary folder and assesses it in one step.

6.4 The capture configuration

Copy `docs/qb-capture.example.yml` to `local/qb-capture.yml`, which is gitignored, and fill it in:

```
schema: inspection-reconcile/qb-capture/v1
realm_hostname: example.quickbase.com
app_id: bxxxxxxx
token_env: QB_USER_TOKEN
page_size: 1000
capture_files: true
scope:
  scope_revision: S1
  accepted: {by: "Name of the person who accepted the scope", at: "2026-10-01T00:00:00Z",
reference: "REF-1"}
limits:
  max_file_bytes: 104857600
  requests_per_10s: 90
  max_attempts: 5
  max_retry_wait_s: 60
  max_pages: 10000
operator_attestation:
  full_read_access: true
  attested_by: "Name of the person attesting"
  note: "Authorized test app; the token's role can read every record of the mapped tables"
```

member	rule	meaning
<code>realm_hostname</code>	a host name	your realm, such as <code>example.quickbase.com</code>
<code>app_id</code>	a Quickbase ID	the app to read
<code>token_env</code>	a variable name	the environment variable that holds the user token
<code>page_size</code>	1 to 10,000	records per query page
<code>capture_files</code>	boolean	<code>false</code> lists the files without downloading them; evidence coverage is then <code>partial</code>
<code>scope</code>	revision and acceptance	the accepted scope this capture represents
<code>limits.max_file_bytes</code>	1 to $2^{31} - 1$	larger files are recorded as <code>too_large</code> and not kept
<code>limits.requests_per_10s</code>	1 to 100	the client's rate limit; Quickbase allows 100 per 10 seconds per token
<code>limits.max_attempts</code>	1 to 20	attempts per request before the run fails

member	rule	meaning
<code>limits.max_retry_wait_s</code>	0 to 3,600	the longest wait a <code>Retry-After</code> header can impose
<code>limits.max_pages</code>	1 to 1,000,000	a safety stop per table
<code>operator_attestation</code>	name, note, <code>full_read_access</code>	a person's statement that the token's role reads every record of the mapped tables

6.5 Tokens and least privilege

1. Create a role, for example **Reconcile Reader**, with **view** permission on the mapped tables, no add, modify or delete, and no admin rights.
2. Assign it to the user who will own the token, and create a user token for that user, assigned to this app only.
3. Put the token in the environment variable named by `token_env`. In PowerShell:

```
$env:QB_USER_TOKEN = Read-Host -MaskInput "Quickbase user token"
```

Then run the capture:

```
uv run inspection-reconcile capture-quickbase --config local/qb-capture.yml \
  --mapping local/quickbase-live.yml --out local/capture
```

Caution

Never put a token in a file, on a command line, or in a URL. The tool reads it only from the environment, never writes it anywhere, and masks it in every log record and error message. A test scans every output for it.

Attest `full_read_access: true` only if the role really reads every record. The attestation is one of the three bases the demonstration pack requires before it accepts a live capture as complete.

6.6 How a capture reads

1. **Schema verification.** For every mapped table, `GET /fields`. Every mapped field must exist with the expected field type and mode; otherwise the run stops with `QB_SCHEMA_MISMATCH`, naming both values.
2. **Pass 1: full-table reads.** Records are read in Record ID order with *keyset* paging (`{3.GT.n}`). Every page's reported total must agree with the first, and IDs must strictly increase. A selected field missing from a response is `FIELD_NOT_RETURNED`, which usually means a permission problem.
3. **The skip fallback.** If a realm refuses the keyset filter on a table's first query (HTTP 400), that table is read with `skip` paging instead, with the same accounting. The capture manifest records which mode each table used.

4. **Files.** For each current artifact of an in-scope inspection, the highest file version is downloaded, decoded, size-checked and hashed. The download stops as soon as a file exceeds `max_file_bytes`.
5. **Pass 2.** After every table has been read once, each table's record IDs and modification times are read again. Any difference means the source changed during the capture, and the coverage claim is weakened accordingly.
6. **Coverage declarations.** A table is declared complete, on the basis `query_total_matched`, `two_pass_stable` and `operator_attestation`, only when all three were earned. Otherwise it is `partial` or `unverified`.
7. **The obligations table must be complete and stable.** The expected work must be reliable, so otherwise the run fails (`QB_SCOPE_UNSTABLE`).

The capture writes only into `--out`. If anything fails, the files it wrote are removed. With `--force`, a previous capture in `--out` is replaced only when the new one succeeds.

6.7 Rate limits and retries

situation	behavior
more than <code>requests_per_10s</code> requests in a sliding 10-second window	the client waits; the default is 90, under Quickbase's limit of 100
HTTP 429	waits for <code>Retry-After</code> , capped at <code>max_retry_wait_s</code> , then retries
HTTP 5xx, connection errors, timeouts	backs off 1, 2, 4, 8 ... seconds (capped at 30), then retries
HTTP 401 or 403	stops: <code>QB_PERMISSION</code> , naming the operation, never the token
any other 4xx	stops: <code>QB_HTTP_ERROR</code> , never retried
<code>max_attempts</code> reached	stops: <code>QB_RETRIES_EXHAUSTED</code>

Connections time out after 10 seconds, and reads after 60. Requests are sequential.

6.8 Live verification

[docs/c3-runbook.md](#) turns Appendix D of the specification into commands. It covers:

1. building a four-table test app whose field IDs reproduce the demonstration mapping;
2. importing the synthetic North Creek data;
3. creating the read-only role and the token;
4. capturing, assessing, and comparing the result with the canonical scenario S01.

The expected result is `READY_FOR_REVIEW`, with no finding added, removed or changed in outcome or reason.

Note

A tool that builds the Appendix D test app through the Quickbase API is in development. It will be the only code in the repository that writes to Quickbase, and it will live in `tools/`, separate from the read-only package. Until it lands, the runbook's manual steps apply.

7 Security model

To report a vulnerability, follow [SECURITY.md](#): reports are private, never public issues.

7.1 Read-only by construction

The capture client can issue exactly four operations:

- `GET /fields`;
- `GET /tables/{id}`;
- `POST /records/query`, which is a read by its documented semantics;
- `GET /files/{table}/{record}/{field}/{version}`, with a version of at least 1.

Anything else is refused before any network I/O. `DELETE` cannot be issued. Nothing in the package writes to Quickbase or to any other source system.

7.2 Tokens

Tokens are read only from the configured environment variable. They are never accepted as an argument, written to disk, put in a URL or logged. A logging filter and exception sanitizing replace the token, and any

`Authorization` value, with `***`.

7.3 Input parsing

- **YAML** goes through the restricted safe loader described in [Safe YAML](#).
- **JSON configuration** is validated closed-world, member by member.
- **CSV** is read with the standard library, with strict UTF-8.

7.4 Files and paths

- **Paths** must satisfy the PATH grammar; a path that does not is never opened.
- **Resolution** is exact-case, segment by segment, on every operating system.
- **Links** (symbolic links, junctions, reparse points) are never followed, and nothing outside the evidence root is read.
- **Size and hashing**. Files above `max_file_bytes` are not hashed. Hashing is streamed in 1 MiB chunks.

7.5 The HTML report

`report.html` is rendered with automatic escaping, so a hostile value in a source record is shown as text, never run as markup. It contains no JavaScript and loads no external resources. A Content-Security-Policy meta tag (`default-src 'none'; style-src 'unsafe-inline'`) blocks everything else, even if a page were modified.

7.6 Outputs

- Nothing is written outside `--out`, and every file is written atomically: to a temporary name, flushed, then renamed.
- An existing output is protected unless `--force` is given. Even then, only the command's own output names are replaced. `assess` and `demo` delete their previous reports and write the new ones. `normalize` and `capture-quickbase` build the new output in a staging folder inside `--out` and swap it in all-or-nothing, only after the run succeeded, so a failed run leaves the previous output exactly as it was.
- If a write fails, the files and folders this run created are removed.

7.7 Data handling

The repository contains synthetic data only, and example emails use the reserved `.invalid` domain. Keep real captures, real mappings and configuration in the gitignored `local/` folder. Before sharing a real capture, replace the realm host name, the app and table IDs, and every user name and email.

8 Outputs and provenance

8.1 assessment.json

The complete, machine-readable result: the identities, the status, the evaluation time, the project, the scope, the pack, the snapshot, the coverage table, the counts and every finding. Members always appear in the same order, and the file contains no wall-clock time and no local path. For the same inputs it is byte-identical on every operating system.

8.2 run-manifest.json

The audit record of one run: the tool and Python versions, the platform, when it ran, the pack, and every input file that was read, with its role, path, size and SHA-256 digest. It is the only output that records when and where a run happened. Input paths are relative to the current folder when possible.

For `assess --export`, the inputs also include the export's `capture-manifest.json` (role `export_manifest`), each table file `normalize read` (`export_table`) and the mapping file (`mapping`). The normalized snapshot's own files are listed with a `null` path, because their temporary folder is removed after the run.

8.3 Identities and determinism

identity	covers	use
<code>evaluation_id</code>	the engine version, the pack, the snapshot's content and the evaluation time	reproducibility: shuffled rows, or an export of the same data, give the same id
<code>assessment_semantic_sha256</code>	every finding's key, outcome, reason, values and dependencies	<code>compare</code> ; two runs with equal values have equal findings
<code>provenance_id</code>	the evaluation, plus the digests of every input byte that was read	audit: exactly which input files were read
snapshot semantic digest	the snapshot's content, excluding provenance details	the snapshot's identity
policy digest	the parsed pack	the pack's identity, independent of formatting

All digests are SHA-256 over canonical JSON (RFC 8785).

8.4 comparison.json

`compare` matches findings by key and writes `added`, `removed`, `changed` (with each finding's outcome and reason before and after) and `unchanged_count`. It exits 20 when anything changed, so it serves as a regression check.

8.5 The SQL export and cross-check

`export-sqlite` writes the snapshot's valid rows to a SQLite file:

```
uv run inspection-reconcile export-sqlite \  
  --snapshot fixtures/scenarios/S01-clean/snapshot \  
  --policy policies/north-creek-demo.yml --out out/s01.sqlite
```

The queries in `sql/` re-derive five checks in plain SQL: duplicate keys, inspection cardinality, identity mismatches, missing document kinds and unmatched inspections. The test suite checks that they agree with the engine on every applicable scenario. They are useful for ad-hoc analysis in any SQL tool. For example:

```
-- current inspections that point outside the accepted scope (R7)  
SELECT DISTINCT inspection_id  
FROM inspections  
WHERE is_current = 1  
      AND (obligation_id IS NULL OR obligation_id NOT IN (SELECT obligation_id FROM obligations));
```

9 Operations

9.1 Command reference

command	purpose	options
<code>assess</code>	assess a snapshot or an export	<code>--snapshot DIR</code> or <code>--export DIR</code> <code>--mapping FILE</code> ; <code>--policy FILE</code> ; <code>--out DIR</code> ; <code>--as-of TS</code> ; <code>--force</code>
<code>validate</code>	check the configuration and list record defects	<code>--snapshot DIR</code> <code>--policy FILE</code>
<code>compare</code>	compare two assessments by finding key	<code>--before PATH</code> <code>--after PATH</code> ; <code>--out FILE</code> ; <code>--force</code>
<code>normalize</code>	turn a Quickbase-shaped export into a snapshot	<code>--export DIR</code> <code>--mapping FILE</code> <code>--out DIR</code> ; <code>--force</code>
<code>capture-quickbase</code>	read a Quickbase app into an export (read-only)	<code>--config FILE</code> <code>--mapping FILE</code> <code>--out DIR</code> ; <code>--force</code>
<code>evidence-digest</code>	print the digest an approval should store	<code>--snapshot DIR</code> <code>--policy FILE</code> <code>--inspection ID</code> ; <code>--revision REV</code>
<code>export-sqlite</code>	write the snapshot's valid rows to SQLite	<code>--snapshot DIR</code> <code>--policy FILE</code> <code>--out FILE</code> ; <code>--force</code>
<code>demo</code>	run the fixture scenarios against the oracle	<code>--scenario ID</code> or <code>--all</code> ; <code>--out DIR</code> ; <code>--fixtures DIR</code> ; <code>--force</code>

Every command accepts `--log-json`, before or after the command name. `--version` and `--help` work as usual.

environment variable	effect
<code>QB_USER_TOKEN</code> (or the name in <code>token_env</code>)	the Quickbase user token, for <code>capture-quickbase</code>
<code>INSPECTION_RECONCILE_DEBUG=1</code>	adds a traceback to <code>INTERNAL_ERROR</code> diagnostics
<code>INSPECTION_RECONCILE_BROWSER</code>	the browser that <code>tools/make_screenshots.py</code> uses

9.2 Exit codes

exit	meaning
0	<code>assess</code> : READY_FOR_REVIEW. <code>compare</code> : no semantic difference. <code>demo</code> : every scenario matched. Any other command: success
10	<code>assess</code> : BLOCKED
11	<code>assess</code> : UNKNOWN
2	run error: usage, configuration, unreadable input, or an internal error caught at the top level
20	<code>compare</code> : semantic differences found
30	<code>demo</code> : at least one scenario did not match its oracle

`demo --scenario ID` exits 30 on an oracle mismatch, and otherwise with that scenario's status code. `assess` writes its outputs before exiting 10 or 11. Exit code 1 is never used deliberately.

9.3 Run-error codes

A run error prints one diagnostic, `error: CODE: message`, writes nothing, and exits 2. The **verified** column says how each code was confirmed for this guide: **CLI** means it was triggered through the command line, **tests** means the test suite exercises it, and **inspection** means it was confirmed by reading the source only.

code	meaning	what to do	verified
USAGE	a missing or conflicting option: <code>--export</code> without <code>--mapping</code> , <code>--mapping</code> with <code>--snapshot</code> , or an <code>--as-of</code> without a time zone offset	correct the command	CLI
OUT_NOT_EMPTY	<code>--out</code> already holds files. For <code>normalize</code> and <code>capture-quickbase</code> with <code>--force</code> : it holds something other than their own output	use an empty folder, add <code>--force</code> , or move the other files away	CLI
OUT_INVALID	<code>--out</code> exists but is not a folder, or <code>compare --out</code> names a folder	name a folder (or, for <code>compare</code> , a file)	CLI
OUT_EXISTS	the <code>--out</code> file of <code>compare</code> or <code>export-sqlite</code> exists	choose another name, or add <code>--force</code>	CLI
COMPARE_OUT_IS_INPUT	<code>compare --out</code> names one of the assessments being compared	write the comparison elsewhere	CLI
COMPARE_INPUT_INVALID	a <code>compare</code> argument is not a readable assessment	point it at an <code>assessment.json</code> or an output folder	CLI
WRITE_FAILED	an output could not be written (disk full, permissions)	fix the cause and re-run; everything this run wrote was removed	tests
CONFIG_INVALID	a pack, mapping, capture configuration, manifest, project or normalization file breaks its schema	read the message: it names the file and the member	CLI

code	meaning	what to do	verified
<code>CONFIG_UNREADABLE</code>	a YAML file cannot be read	check the path and permissions	CLI
<code>POLICY_UNREADABLE</code>	the pack file cannot be read	check the path and permissions	CLI
<code>POLICY_NOT_EFFECTIVE</code>	the evaluation time is before the pack's <code>effective_from</code>	use the pack in force at that time, or another <code>--as-of</code>	CLI
<code>PROJECT_MISMATCH</code>	<code>project.json</code> names another project than the pack	use the pack for this project	CLI
<code>SNAPSHOT_MISSING</code>	<code>--snapshot</code> is not a folder	check the path	CLI
<code>INPUT_UNREADABLE</code>	<code>manifest.json</code> or <code>project.json</code> cannot be read	check the files and permissions	tests
<code>DATASET_MISSING_ON_DISK</code>	a dataset or the evidence folder that the manifest declares is absent	restore it, or remove it from the manifest	CLI
<code>DATASET_UNREADABLE</code>	a declared dataset file cannot be opened	check permissions	tests
<code>CSV_UNREADABLE</code>	a CSV file is not UTF-8, contains a NUL byte, is empty, or has an unparseable header	fix the export	CLI
<code>CSV_HEADER_INVALID</code>	a CSV header has an unknown, duplicated or missing column	fix the export's columns	CLI
<code>EVIDENCE_ROOT_INVALID</code>	the evidence root is a link, a junction or not a folder	make it a real folder	CLI
<code>EXPORT_INVALID</code>	a Quickbase-shaped export contradicts its own manifest	capture again; do not edit exports by hand	CLI
<code>UNKNOWN_SCENARIO</code>	<code>demo --scenario</code> names no oracle scenario	check the scenario id	CLI
<code>INSPECTION_NOT_FOUND</code>	<code>evidence-digest</code> : the inspection's revision is not exactly one valid row	check the inspection id and revision	CLI
<code>EVIDENCE_INCOMPLETE</code>	<code>evidence-digest</code> : a required document is missing or its file is unavailable	complete the evidence first	CLI
<code>REQUIREMENT_UNDEFINED</code>	<code>evidence-digest</code> : the pack defines no documents for the activity	add the activity to the pack	tests
<code>QB_TOKEN_MISSING</code>	the token environment variable is unset or empty	set it (see Tokens and least privilege)	CLI
<code>QB_CONFIG</code>	the client's settings are invalid, or the token is malformed	check the configuration; the token is never shown	tests
<code>QB_SCHEMA_MISMATCH</code>	a mapped field is missing, or has another type or mode than the mapping expects	correct the mapping, or the app	tests

code	meaning	what to do	verified
<code>FIELD_NOT_RETURNED</code>	a query response lacks a selected field	check the role's field permissions	tests
<code>QB_PERMISSION</code>	Quickbase answered 401 or 403	check the token, the role and the app assignment	tests
<code>QB_HTTP_ERROR</code>	Quickbase answered another 4xx	read the message; the request is not retried	tests
<code>QB_RETRIES_EXHAUSTED</code>	throttling, server errors or connection errors persisted past <code>max_attempts</code>	re-run later, or raise the limit deliberately	tests
<code>QB_PROTOCOL</code>	a response did not match the documented API shape	report it; the capture refuses to guess	tests
<code>QB_SCOPE_UNSTABLE</code>	the obligations table was incomplete or changed during the capture	capture again when the source is stable	tests
<code>QB_MAX_PAGES</code>	a table needed more than <code>limits.max_pages</code> pages	raise the limit deliberately	tests
<code>QB_CLIENT_CLOSED</code>	an internal error: the client was used after it was closed	report it	tests
<code>NOT_AVAILABLE</code>	an optional component is not installed, for example <code>capture-quickbase</code> without the <code>quickbase</code> extra	<code>uv sync --extra quickbase</code>	inspection
<code>ENGINE_DUPLICATE_KEY</code>	an internal invariant failed: two findings share a key	report it	inspection
<code>INTERNAL_ERROR</code>	an unexpected exception, caught at the top level	re-run with <code>INSPECTION_RECONCILE_DEBUG=1</code> and report the traceback	tests

9.4 Diagnostics and `--log-json`

Standard output carries the result: a summary line for `assess`, a line per scenario for `demo`, and so on. Standard error carries diagnostics only. With `--log-json`, every line on standard error is one JSON object:

```
{"level": "error", "code": "OUT_NOT_EMPTY", "message": "out/s02: exists and is not empty (use --force)"}
```

Log records from library modules, such as the capture's warnings, use the same shape with `"code": "LOG"`, plus a `logger` member.

9.5 Automation

- **Scheduling.** Run `capture-quickbase`, then `assess --export`, and act on the exit code: 0 ready, 10 blocked, 11 unknown, 2 run error.

- **Regression checks.** Keep the last good `assessment.json`, and run `compare` against each new one. Exit 20 means something changed; `comparison.json` says what.
- **Archiving.** Keep `run-manifest.json` with each result: together with the inputs it lists, it reproduces the assessment exactly.

9.6 Continuous integration and the demonstration site

The CI workflow runs the linters, the type checker, the full test suite and `demo --all` on Linux, Windows and macOS, each with Python 3.12 and 3.13. A final job checks that every platform produced identical evaluation identities and report digests.

The demonstration site, bochen2029-pixel.github.io/inspection-reconcile, publishes the output of `demo --all`: every scenario's report and the index. It is deployed by a workflow that runs only after CI has passed on a push to the main branch, and only `demo --all` itself decides what is published: any scenario that departs from its oracle fails the deployment.

9.7 Rebuilding the screenshots and these guides

```
uv run python tools/make_screenshots.py
uv run --no-project --with markdown-it-py --with pypdf python tools/build_guides.py
```

The first regenerates every image in `docs/images` from the tool's own output, with headless Chrome or Edge. The second renders `docs/guide/*.md` into these PDFs. Both bound every browser run with a timeout and use a throwaway browser profile.

10 Limitations and verification status

10.1 What is verified

part	status
Engine semantics (R0–R7, coverage, status, identities)	verified against a hand-written oracle of 24 scenarios, independently re-derived from the specification, plus rule-level, fault-injection and property tests
Digests	the specification's test vectors are reproduced; the fixtures' digests come from an independent implementation in the generator
Cross-platform identity	CI compares Linux, Windows and macOS
Quickbase export adapter (normalize)	built from the published contract and synthetic exports; S16 normalizes to S01's exact identities
Quickbase capture client	tested against a mock app that follows the published contract: pagination, rate limits, retries, permission errors, schema drift, mid-capture changes, oversized files and token redaction. Not yet run against a live Quickbase app
The keyset filter <code>{3.GT.n}</code>	not confirmed by Quickbase's documentation; if a realm refuses it, the skip fallback applies. Both modes are tested against the mock

10.2 Known limitations

- **One problem at a time per obligation.** A second, independent problem appears after the first is fixed.
- **Revision binding** cannot detect changed bytes under an unchanged revision label.
- **No transactional snapshot.** Quickbase offers none; the two-pass check detects change but cannot prevent it.
- **Coverage is a claim.** The tool reports it and its basis, but cannot prove that no hidden records exist.
- **A hash proves bytes only.** It proves nothing about authorship, truthfulness, or whether the physical work was done.
- **Fixed cardinality.** Exactly one current, completed inspection per obligation; repeat inspections need a supersession rule.
- **Full-table capture.** Scoped capture of very large tables is specified but not implemented.
- **A trusted evidence root.** The evidence folder is assumed to be a local folder that only the operator controls.

10.3 What is never claimed

That the tool is production-ready; that it establishes compliance; that any organization uses it; or any runtime, accuracy or labor-saving figure that was not measured.